

PROGETTO SDCC B1

Service Registry Distribuito su istanza EC2

I. INTRODUZIONE E OBIETTIVI

In un sistema composto da microservizi, le istanze applicative possono nascere, terminare o cambiare collocazione durante l'esecuzione. Un catalogo statico di indirizzi non è quindi in grado di rappresentare in modo affidabile lo stato corrente dell'infrastruttura. È pertanto necessario un componente capace di raccogliere le registrazioni, verificarne la vitalità e restituire ai client un endpoint aggiornato.

Questo progetto sviluppa in Go un **registro di servizi distribuito**. Il registro conserva il nome, l'endpoint e lo stato di salute delle istanze applicative. La sua caratteristica centrale è l'assenza di un punto di controllo unico: cinque nodi registry condividono le informazioni attraverso un protocollo di gossip e sincronizzazioni periodiche.

Gli obiettivi sono dimostrare la discovery decentralizzata, la convergenza delle viste locali, la continuità del servizio dopo il guasto di uno o più nodi e il recupero automatico dello stato. L'intero ciclo è riproducibile tramite Docker Compose e lo script operativo `scripts/dev.ps1`.

II. REQUISITI

Il sistema è pensato per un ambiente distribuito e parzialmente isolato, rappresentato localmente tramite **Docker**. Docker Compose crea una rete condivisa nella quale i nodi possono risolversi attraverso i rispettivi nomi DNS. Ogni registry vive in un container separato e mantiene una composta dal catalogo dei servizi e dalla membership dei peer. I cinque nodi ricevono configurazioni differenti per quanto riguarda l'identificativo e l'indirizzo pubblicizzato. Non vengono utilizzati volumi Docker per la persistenza locale. Per rendere l'esecuzione ripetibile, l'ambiente host deve disporre dei seguenti strumenti:

- **Docker Desktop**: gestisce immagini, container, rete virtuale e mappatura delle porte gRPC.
- **PowerShell**: esegue `scripts/dev.ps1`, che raccoglie i comandi del progetto e automatizza lo scenario.
- **Go 1.24 o successivo**: permette di compilare il codice e lanciare la suite di test sull'host.

Per eseguire il progetto si dispone di una sequenza di comandi forniti dallo script `scripts/dev.ps1`, che riproduce l'intero ciclo di vita di un servizio all'interno del cluster. Il flusso inizia con il comando `up`, che ricrea l'ambiente, prepara le immagini Docker e avvia i cinque registry. Successivamente, `select-service` seleziona il profilo del servizio e `register` invia il relativo record tramite la CLI containerizzata. Il record viene quindi diffuso dal

meccanismo di gossip, consentendo ai peer di convergere progressivamente verso uno stato comune. Il registry conserva e replica le informazioni del servizio senza controllarne direttamente l'endpoint. La fase di fault injection arresta casualmente dei nodi registry, simulando un guasto parziale del cluster; i peer superstiti continuano tuttavia a rispondere alle richieste e a mantenere le copie replicate. Infine, i nodi arrestati vengono riavviati e recuperano lo stato tramite gossip e reconcile, mentre `deregister` propaga la rimozione logica del servizio e `down` arresta e rimuove l'ambiente Docker.

III. ARCHITETTURA DEL SISTEMA

Il sistema adotta un modello peer-to-peer nel quale ogni registry mantiene una copia locale del catalogo e della membership. I nodi non eleggono un coordinatore permanente: ciascuno può ricevere richieste esterne, propagare aggiornamenti e recuperare informazioni dagli altri peer.

I requisiti principali sono:

- la discovery deve continuare a rispondere anche quando una parte del cluster non è raggiungibile, evitando una dipendenza da un singolo server.
- le modifiche devono raggiungere progressivamente tutti i nodi mediante gossip, senza introdurre un coordinamento centrale o un consenso sincrono.
- un peer riavviato deve poter ricostruire la membership e i dati persi sfruttando i seed peer, il gossip e il reconcile.
- build, test, deployment e fault injection devono essere eseguibili tramite Docker Compose e un runner unico.

L'architettura espone due interfacce gRPC. Una gestisce il rapporto tra i servizi applicativi e il catalogo, mentre l'altra gestisce la membership e la replica tra i registry. I messaggi sono definiti con Protocol Buffers e trasformati in codice Go tipizzato, riducendo le ambiguità nel contratto di rete e il costo della serializzazione.

IV. gRPC

I contratti di comunicazione sono descritti con **Protocol Buffers (v3)** e compilati in stub Go. Il file `proto/registry.proto` separa le operazioni rivolte ai servizi applicativi da quelle necessarie alla cooperazione tra i registry. Il **ServiceRegistry** governa il ciclo di vita delle istanze che utilizzano la discovery. Espone le seguenti procedure remote:

- **ServiceRegister**: inserisce un'istanza con i relativi dati di rete e metadati.

- **ServiceDeregister**: avvia la rimozione controllata di un'istanza.
- **ServiceGet / List**: permettono di risolvere una singola istanza o di consultare il catalogo locale.

Il **RegistryPeer** definisce le RPC con cui i nodi condividono membership e dati:

- **JoinNode**: annuncia un nodo ai seed peer e recupera la membership conosciuta.
- **GossipUpd**: invia aggiornamenti locali a un insieme limitato di peer.
- **AntyEntropyPull**: recupera lo stato di un peer durante la riconciliazione o il recovery.

Il record principale scambiato tra client e peer è **ServiceMessage**. I principali campi che compongono il messaggio sono strutturati come segue:

- **service_name** (string): nome identificativo e leggibile del servizio, ad esempio **identity-api** o **billing-api**. È il riferimento principale utilizzato durante la discovery.
- **endpoint** (string): indirizzo host e porta ai quali il client può collegarsi.
- **health_status** (enum): stato osservato dell'istanza, ad esempio **serving** o **degraded**.
- **lamport_clock** (uint64): contatore Lamport scalare utilizzato per ordinare gli aggiornamenti del servizio.
- **lamport_node_id** (string): identificativo del nodo utilizzato come criterio di risoluzione dei conflitti a parità di **lamport_clock**.

La versione complessiva di un aggiornamento è quindi determinata dalla coppia

(**lamport_clock**, **lamport_node_id**).

Prima viene confrontato il contatore Lamport; in caso di parità, viene utilizzato l'identificativo del nodo come criterio di ordinamento deterministico.

V. FAULT TOLERANCE

La fault tolerance del progetto combina ridondanza, monitoraggio e riallineamento. I cinque registry conservano viste locali, ma non dipendono da un coordinatore permanente; il guasto di uno o più container può quindi ridurre la conoscenza locale senza rendere indisponibile la discovery sui peer rimasti attivi.

Il sistema controlla esclusivamente la raggiungibilità dei peer registry. La disponibilità dei servizi applicativi non viene verificata dal registro: il servizio responsabile decide autonomamente quando registrarsi o deregistrarsi.

Per i registry, il runtime aggiorna periodicamente l'heartbeat locale e registra l'ultimo aggiornamento ricevuto da ogni peer. Superato **peer_timeout**, il peer non raggiungibile viene eliminato dal **PeerStore** attivo e non viene più selezionato per le comunicazioni gossip o reconcile. Le registrazioni vengono diffuse con **GossipUpd** verso un insieme limitato di peer. Il ciclo *reconcile* interroga

periodicamente un altro nodo tramite **AntyEntropyPull**, recuperando le modifiche perse durante un errore, un riavvio o un isolamento temporaneo.

La coppia

(**lamport_clock**, **lamport_node_id**)

ordina le mutazioni. Il merge applica soltanto i record più recenti dello stato locale, rendendo idempotenti i messaggi duplicati e innocui quelli ricevuti fuori ordine. La fault injection può arrestare casualmente uno o più registry, fino a $N - 2$ nodi del cluster, lasciando almeno due nodi attivi. I peer superstiti continuano a rispondere alle richieste e rimuovono i nodi arrestati dopo il timeout. Al riavvio, il processo Go ricostruisce membership e record tramite seed peer, gossip e reconcile.

Il protocollo distingue un crash da uno shutdown ordinato. Con **GracefulLeave**, il nodo comunica l'uscita ai peer, che lo rimuovono senza attendere il timeout, riducendo il tempo di convergenza. La deregistrazione non elimina immediatamente il record. Il registry crea un *deletion marker*, svuota l'endpoint, assegna **NOT_SERVING** e incrementa **lamport_clock**. Il marker viene quindi replicato come una mutazione più recente e impedisce il ritorno di copie obsolete.

Dopo un intervallo configurabile, il deletion marker scade e viene rimosso dal registro locale. Le normali operazioni di lista e discovery non mostrano i deletion marker.

Il client prova gli endpoint disponibili e tratta l'errore di un singolo peer come un'indisponibilità locale. Le richieste possono quindi proseguire sui nodi superstiti, realizzando una degradazione controllata.

VI. ORCHESTRAZIONE

Docker Compose gestisce i container e **scripts/dev.ps1** offre un'interfaccia unica per le operazioni di sviluppo. Il ciclo operativo distingue il reset iniziale, l'avvio dei nodi e la successiva convergenza delle viste locali.

Il comando **scripts/dev.ps1 up** esegue una sequenza deterministica di preparazione dell'ambiente.

Le operazioni principali sono:

- **docker compose down** elimina i container e la rete della sessione precedente.
- vengono costruite le immagini dei cinque registry e del client CLI.
- viene creato il file locale che segnala l'avvio dello scenario.
- i cinque nodi registry vengono avviati e il runtime gossip avvia automaticamente bootstrap, gossip, reconcile e membership sweep.

La chiusura ordinaria del sistema viene eseguita tramite il comando **scripts/dev.ps1 down**, che arresta e rimuove i container Docker e la rete associata. Poiché il progetto non utilizza volumi per la persistenza, lo stato locale viene ricostruito dai nodi peer quando il cluster viene riavviato.

Il file `deploy/docker-compose.yml` definisce sei servizi collegati alla stessa rete virtuale Docker: cinque registry e un client CLI temporaneo. I cinque registry utilizzano la stessa immagine applicativa, ma si distinguono grazie ai rispettivi file di configurazione YAML. All'interno dei container, ogni registry utilizza la porta gRPC 10001. Per consentire l'accesso dai sistemi esterni, questa porta viene associata a una porta diversa sull'host: il primo nodo usa la porta 10001, il secondo la 10002, il terzo la 10003, il quarto la 10004 e il quinto la 10005. La rete Docker permette ai container di comunicare tra loro utilizzando i rispettivi hostname. Il servizio `service-cli` non viene eseguito continuamente: viene avviato solo quando è necessario eseguire un comando e viene rimosso al termine dell'operazione.

All'avvio di Go, il processo segue quattro passaggi principali:

- 1) il flag `-config` seleziona il profilo YAML del nodo.
- 2) viene aperto il listener e il nodo viene inizializzato con uno store locale vuoto.
- 3) vengono registrate le API per client e peer sullo stesso server.
- 4) viene avviato il runtime gossip, che registra il nodo nella membership locale e avvia i cicli di gossip, reconcile e membership sweep.

L'inizializzazione della membership locale è centralizzata nel metodo `Runtime.Start()`, evitando registrazioni duplicate durante il bootstrap applicativo. Il gossip non richiede un'azione successiva del client: i nodi iniziano a scambiarsi periodicamente lo stato fin dal loro avvio.

Alla ricezione di SIGINT o SIGTERM, il processo arresta i runtime, comunica il proprio *graceful leave* ai peer e termina il server gRPC in modo ordinato.

Il runtime avvia automaticamente le attività di sincronizzazione tra i cinque registry quando i container vengono avviati. Non è presente alcuna barriera esterna che richieda la creazione di file o l'esecuzione di comandi preliminari: il bootstrap, il gossip, il reconcile e il controllo periodico dei peer vengono gestiti direttamente dal runtime.

Il ciclo di bootstrap contatta i peer iniziali indicati nella configurazione e recupera le informazioni già disponibili. Il ciclo di gossip propaga periodicamente i record dei servizi verso un numero limitato di peer, secondo il fan-out configurato. Il ciclo di reconcile esegue invece il recupero periodico dello stato dai nodi conosciuti, permettendo di correggere eventuali differenze tra le repliche.

Parallelamente, il peer delete aggiorna la presenza del nodo locale, rimuove i peer che non risultano più raggiungibili entro il timeout configurato e provvede alla scadenza dei deletion marker. Il parametro `gossip_start_delay_seconds` consente soltanto di ritardare temporalmente l'avvio dei cicli, senza introdurre una barriera basata su file.

Nel deployment Docker corrente, quindi, la propagazione dello stato inizia automaticamente all'avvio dei container e non dipende dall'esecuzione del comando `register`.

Il container `service-cli` esegue comandi brevi nella rete Docker tramite `docker compose run -rm service-cli`. Le operazioni disponibili sono:

Se uno o più peer sono spenti, il client riconosce gli errori di disponibilità e segnala il nodo disattivato invece di interrompere il processo con un'eccezione non gestita. Il runner PowerShell permette inoltre di distinguere la build iniziale dai successivi comandi CLI, evitando ricostruzioni non necessarie.

Il modulo `ServiceStore` rappresenta in memoria lo stato locale delle istanze. Ogni modifica passa attraverso normalizzazione, validazione e versionamento Lamport, così da rendere il comportamento prevedibile anche quando gli aggiornamenti arrivano in modo asincrono.

La versione di un aggiornamento è determinata dalla coppia

`(lamport_clock, lamport_node_id)`,

utilizzata per ordinare i record e risolvere deterministicamente i conflitti tra aggiornamenti concorrenti. Per evitare che un vecchio messaggio faccia riapparire un servizio rimosso, il progetto utilizza un **deletion marker**:

- `DeregisterService` svuota l'endpoint del servizio;
- lo stato viene impostato su `NOT_SERVING`;
- `lamport_clock` viene incrementato per dare priorità alla cancellazione;
- `deletion_marker_expires_at_unix` stabilisce il momento oltre il quale il marker può essere rimosso.

La cancellazione viaggia quindi come un normale aggiornamento, ma con una versione maggiore. I peer la applicano durante il merge e non accettano record precedenti rispetto alla coppia. Il deletion marker viene mantenuto abbastanza a lungo da consentirne la propagazione e viene successivamente eliminato dalla procedura periodica di pulizia.

La disponibilità dell'endpoint applicativo non viene verificata dal registry. La rimozione del servizio avviene esplicitamente tramite `DeregisterService`; il record trasformato in deletion marker viene quindi replicato agli altri nodi.

La componente `Runtime Gossip` realizza la sincronizzazione decentralizzata. Invece di bloccare l'intero cluster con un consenso sincrono, esegue loop indipendenti, ciascuno dedicato a una responsabilità specifica e regolato dal proprio intervallo temporale. All'avvio e a intervalli regolari invia `JoinNode` ai seed peer configurati. Il nodo si annuncia, viene inserito nella membership remota e riceve l'elenco aggiornato dei peer da aggiungere al proprio `PeerStore`. Seleziona pseudocasualmente fino a k peer dal `PeerStore` e invia loro un messaggio `GossipUpd`. L'informazione si diffonde in più direzioni senza richiedere che ogni nodo conosca immediatamente l'intera topologia. opera più lentamente del gossip e interroga un peer tramite `AntyEntropyPull`. Il parametro `since_unix` consente di richiedere gli aggiornamenti successivi all'ultimo allineamento, recuperando le informazioni perse durante errori,

riavvii o partizioni temporanee. Aggiorna periodicamente l'heartbeat del nodo locale e controlla l'età dell'ultima interazione con ciascun peer. Se viene superato il limite `peer_timeout_`, il peer viene rimosso dalla membership attiva e non viene più scelto per le comunicazioni successive. Lo stesso loop elimina anche i deletion marker scaduti. La consistenza eventuale nasce dalla combinazione di tre strategie complementari:

- 1) diffondere rapidamente registrazioni, aggiornamenti e deletion marker verso più peer, fornendo il percorso principale di replica durante il funzionamento normale.
- 2) colmare i vuoti lasciati da messaggi persi, congestione o riavvii, permettendo al nodo di recuperare lo stato dal peer interrogato.
- 3) mantenere un punto di rientro nella rete quando la membership locale diventa incompleta o un nodo torna operativo dopo una separazione temporanea.

VII. STRATEGIA DI TEST E VALIDAZIONE

La validazione comprende test unitari, test sui server gRPC, test d'integrazione e prove Docker. I test unitari verificano le regole applicative, la validazione, il versionamento Lamport e i deletion marker; i test gRPC coprono registrazione, discovery, deregistrazione, idempotenza e membership.

I test d'integrazione avviano nodi su porte temporanee e controllano la convergenza dei servizi, il pruning dopo un crash, il recupero dei dati dai peer dopo il riavvio, la propagazione della deregistrazione e il graceful leave senza attesa del timeout. Lo scenario multi-crash verifica inoltre l'arresto e il successivo recupero di più nodi.

La suite standard si esegue con:

```
go test ./...
```

Il ciclo containerizzato si riproduce con:

```
.\scripts\dev.ps1 trace-up
.\scripts\dev.ps1 select-service -profile random
.\scripts\dev.ps1 register
.\scripts\dev.ps1 discovery
.\scripts\dev.ps1 deregister
.\scripts\dev.ps1 down
```

La build delle immagini viene eseguita nella fase iniziale; i comandi CLI successivi riutilizzano le immagini già disponibili. Per la fault injection, `scripts/dev.ps1 crash` arresta casualmente un registry se utilizzato senza argomenti, mentre l'opzione `-count` consente di arrestare più nodi, fino a $N - 2$, lasciando almeno due nodi attivi. Il comando `verify-resilience` controlla i peer superstiti e `recover` riavvia tutti i nodi precedentemente arrestati.

VIII. CONCLUSIONI

Il progetto realizza un registro distribuito con cinque nodi cooperanti, API gRPC tipizzate e replica gossip. Il versionamento Lamport, i deletion marker, il pruning

e il reconcile gestiscono la replica dei record e l'arresto temporaneo o permanente di uno o più registry, mentre la disponibilità dei servizi applicativi resta responsabilità dei servizi stessi.

La separazione tra dominio, applicazione, adapter, storage, gossip e bootstrap rende più chiari i confini del codice. Docker Compose e `scripts/dev.ps1` forniscono inoltre un ambiente riproducibile per build, test, discovery e simulazione dei guasti.

Nel complesso, la soluzione mostra come ottenere disponibilità e consistenza eventuale senza un coordinatore centrale, mantenendo strumenti concreti per osservare e verificare il comportamento del cluster, inclusi la propagazione degli aggiornamenti, il recupero dopo i crash e la rimozione dei servizi tramite deletion marker.